

Calculating VaR using Parametric Approach

The Parametric approach, also known as the Variance–Covariance method, is a widely used technique for estimating Value at Risk (VaR). Under this method, portfolio risk is quantified by estimating the volatility of returns, typically measured using the standard deviation derived from historical data or statistical models.

The approach assumes that portfolio returns follow a normal distribution, which allows VaR to be calculated analytically for a given confidence level. By combining the estimated volatility with the appropriate z-score, the method provides a closed-form estimate of the maximum expected loss over a specified time horizon under normal market conditions.

Formula: $VaR = \text{Portfolio Value} * Z\text{-score} * \text{Portfolio Volatility} * (\text{Time Horizon})^{0.5}$

```
In [ ]: # Importing Required Libraries
import numpy as np
import pandas as pd
import datetime as dt
import yfinance as yf
import matplotlib.pyplot as plt
from scipy.stats import norm

years=15

end_date= dt.datetime.now()
start_date= end_date - dt.timedelta(days=365*years)

tickers= ['ICICIBANK.NS', 'HDFCBANK.NS', 'KOTAKBANK.NS', 'AXISBANK.NS', '^NSEI']
```

```
In [ ]: adj_close_df= pd.DataFrame()
for ticker in tickers:
    data= yf.download(ticker, start=start_date, end=end_date)
    adj_close_df[ticker]= data['Close']

log_returns= np.log(adj_close_df/adj_close_df.shift(1))
log_returns= log_returns.dropna()
```

```
In [3]: portfolio_value= 1000000
weights= np.array([1/len(tickers)] * len(tickers))
```

```
historical_returns= (log_returns*weights).sum(axis=1)
```

```
In [4]: days = 5
historical_x_day_returns = historical_returns.rolling(window=days).sum()
```

```
In [5]: cov_matrix = log_returns.cov() * 252
portfolio_std_dev = np.sqrt(weights.T @ cov_matrix @ weights)
```

```
In [8]: confidence_levels = [0.90, 0.95, 0.99]
```

```
VaRs=[]
for cl in confidence_levels:
    VaR = -portfolio_value * (norm.ppf(1 - cl) * portfolio_std_dev * np.sqrt(days / 252) - historical_returns.mean() * days)
    VaRs.append(VaR)
```

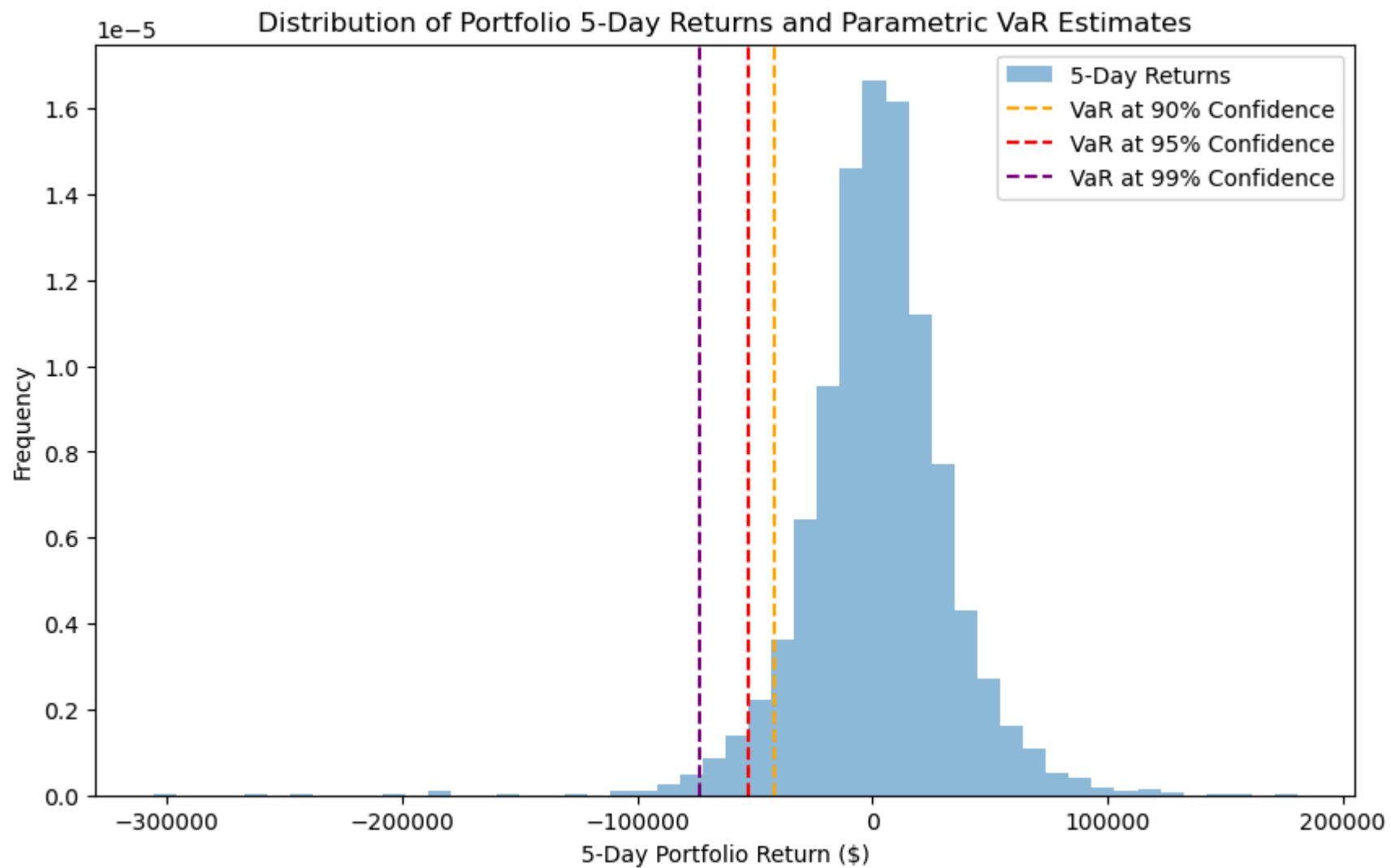
```
In [10]: # Converting returns to dollar values for the histogram
historical_x_day_returns_dollar = historical_x_day_returns * portfolio_value

# Plot the histogram
plt.figure(figsize=(10, 6))
plt.hist(historical_x_day_returns_dollar, bins=50, density=True, alpha=0.5, label=f'{days}-Day Returns')

# Define colours for each confidence interval
color_map={0.90:'orange',0.95:'red',0.99:'purple'}

# Plot VaR Lines
for cl, VaR in zip(confidence_levels, VaRs):
    plt.axvline(x=-VaR, linestyle='--', color=color_map.get(cl, 'black'), label='VaR at {}% Confidence'.format(int(cl * 100)))

plt.xlabel(f'{days}-Day Portfolio Return ($)')
plt.ylabel('Frequency')
plt.title(f'Distribution of Portfolio {days}-Day Returns and Parametric VaR Estimates')
plt.legend()
plt.show()
```



Calculating VaR using Historical Simulation

The Historical Simulation method is a non-parametric approach to estimating Value at Risk (VaR) that relies directly on historical return data rather than assuming any specific statistical distribution. Under this method, past portfolio returns are reordered from worst to best, and the VaR is determined by selecting the appropriate percentile corresponding to the chosen confidence level.

This approach implicitly assumes that future risk patterns will resemble historical market behavior. While it is intuitive and easy to implement, Historical VaR may fail to capture tail risks or extreme market events that are not present in the historical dataset, and it does not explicitly account for changing market dynamics. Despite these limitations, it remains a widely used baseline risk measure in portfolio risk management.

Formula: VaR= -Return at the (100-confidence level) percentile

```
In [1]: #Importing Required Libraries
import numpy as np
import pandas as pd
import datetime as dt
import yfinance as yf
import matplotlib.pyplot as plt
from scipy.stats import norm

years=15

end_date= dt.datetime.now()
start_date= end_date - dt.timedelta(days=365*years)

tickers= ['ICICIBANK.NS', 'HDFCBANK.NS', 'KOTAKBANK.NS', 'AXISBANK.NS', '^NSEI']
```

```
In [ ]: adj_close_df= pd.DataFrame()
for ticker in tickers:
    data=yf.download(ticker,start=start_date,end=end_date)
    adj_close_df[ticker]=data['Close']

log_retruns=np.log(adj_close_df/adj_close_df.shift(1))
log_retruns.dropna()
```

```
In [3]: portfolio_value=1000000
weights= np.array([1/len(tickers)]*len(tickers))
```

```
historical_retruns= (log_retruns*weights).sum(axis=1)
```

```
In [4]: days = 5
range_returns= historical_retruns.rolling(window=days).sum()
range_returns= range_returns.dropna()
```

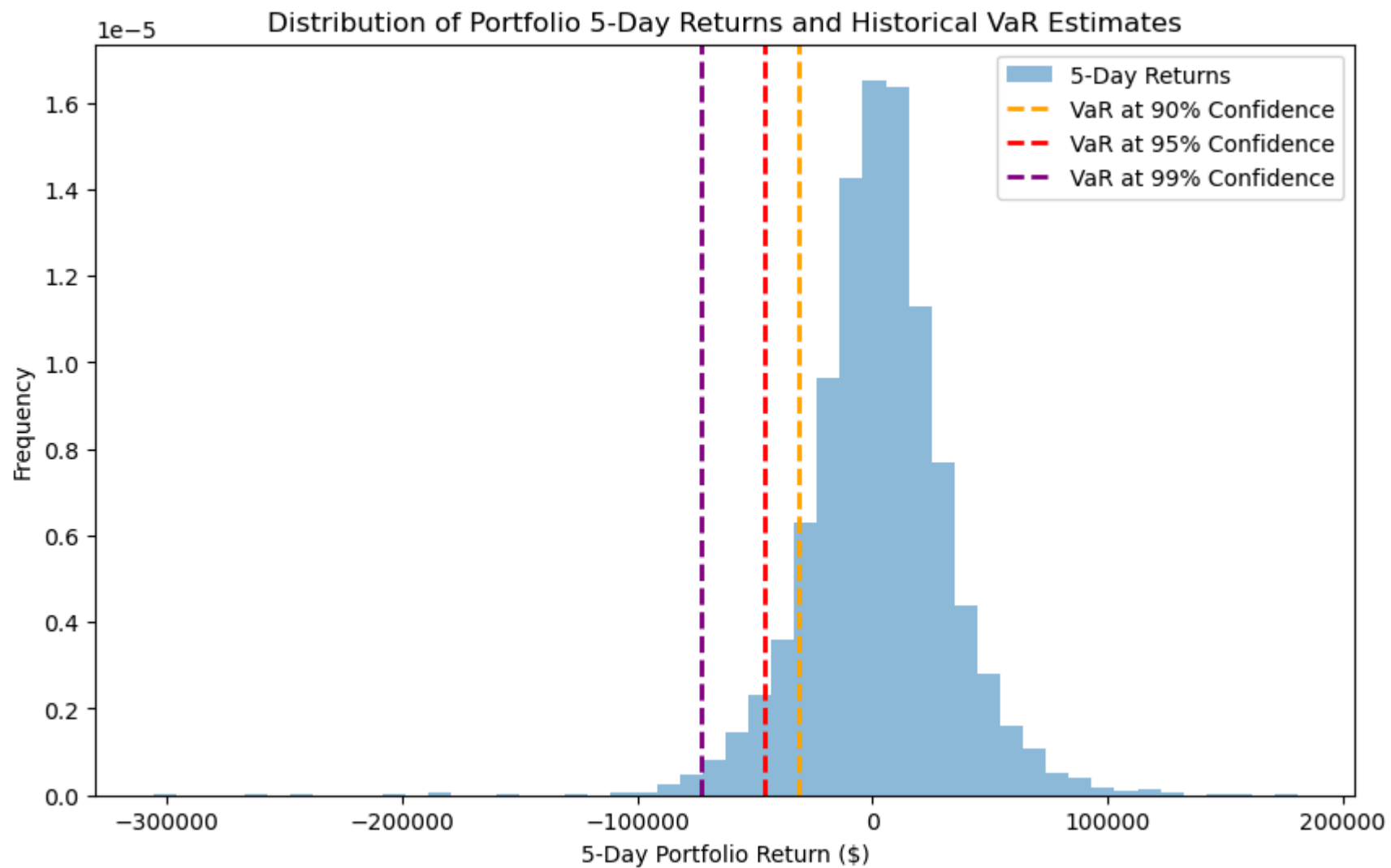
```
In [5]: # Convert returns to dollar values
range_returns_dollar = range_returns * portfolio_value

# Plot histogram
plt.figure(figsize=(10, 6))
plt.hist(range_returns_dollar, bins=50, density=True, alpha=0.5, label=f'{days}-Day Returns')

# Confidence Levels and colors
confidence_levels = [0.90, 0.95, 0.99]
colors = {0.90: 'orange', 0.95: 'red', 0.99: 'purple'}

# Plot VaR Lines
for cl in confidence_levels:
    VaR = -np.percentile(range_returns, (1 - cl) * 100) * portfolio_value
    plt.axvline(x=-VaR, color=colors[cl], linestyle='--', linewidth=2, label=f'VaR at {int(cl * 100)}% Confidence')

plt.xlabel(f'{days}-Day Portfolio Return ($)')
plt.ylabel('Frequency')
plt.title(f'Distribution of Portfolio {days}-Day Returns and Historical VaR Estimates')
plt.legend()
plt.show()
```



Calculating VaR using Monte Carlo Simulation

The Monte Carlo Simulation method is a flexible and forward-looking approach used to estimate Value at Risk (VaR) by modeling the uncertainty in portfolio returns. Under this method, a large number of potential future return scenarios are generated using assumed statistical properties of asset returns, such as expected returns, volatility, and correlations.

By simulating thousands of possible portfolio value paths over a specified time horizon, the distribution of potential losses is obtained. The VaR is then estimated as the quantile of this simulated loss distribution corresponding to the chosen confidence level. Unlike Parametric or Historical VaR, Monte Carlo Simulation can incorporate non-normal return distributions, complex portfolios, and non-linear risk factors, making it a powerful tool for advanced risk analysis.

```
In [4]: # Importing Required Liabraries
import numpy as np
import pandas as pd
import datetime as dt
import yfinance as yf
import matplotlib.pyplot as plt
from scipy.stats import norm

years=15

end_date= dt.datetime.now()
start_date= end_date - dt.timedelta(days=365*years)

tickers= ['ICICIBANK.NS', 'HDFCBANK.NS', 'KOTAKBANK.NS', 'AXISBANK.NS', '^NSEI']
```

```
In [ ]: adj_close_df= pd.DataFrame()
for ticker in tickers:
    data=yf.download(ticker,start=start_date,end=end_date)
    adj_close_df[ticker]=data['Close']

log_retruns=np.log(adj_close_df/adj_close_df.shift(1))
log_retruns.dropna()
```

```
In [9]: # Assuming that future returns are based on past retruns
# Calculating expected protfolio returns

def expected_return(weights,log_returns):
```

```

    return np.sum(log_returns.mean()*weights)

# Calculating portfolio standard deviation

def standard_deviation(weights,cov_matrix):
    variance= weights.T @ cov_matrix @ weights
    return np.sqrt(variance)

cov_matrix= log_retruns.cov()

```

```

In [10]: portfolio_value = 1000000
weights = np.array([1/len(tickers)]*len(tickers))
portfolio_expected_return = expected_return(weights, log_retruns)
portfolio_std_dev = standard_deviation (weights, cov_matrix)

```

```

In [12]: def random_z_score():
    return np.random.normal(0, 1)

days= 5

def scenario_gain_loss(portfolio_value, portfolio_std_dev, z_score, days):
    return portfolio_value*(portfolio_expected_return * days + portfolio_std_dev * z_score * np.sqrt(days))

```

```

In [13]: # Run 10000 simulations

simulations = 10000
scenario_return = []

for i in range(simulations):
    z_score = random_z_score()
    scenario_return.append(scenario_gain_loss(portfolio_value, portfolio_std_dev, z_score, days))

```

```

In [18]: # Confidence Levels and colors

confidence_levels = [0.90, 0.95, 0.99]
colors = {0.90: 'orange', 0.95: 'red', 0.99: 'purple'}

# Calculate Monte Carlo VaR for each confidence Level

VaRs = {}
for cl in confidence_levels:
    VaRs[cl] = -np.percentile(scenario_return, (1 - cl) * 100)

```



```
# Plot histogram of Monte Carlo simulated returns
```

```
plt.figure(figsize=(10, 6))
```

```
plt.hist(scenario_return, bins=50, density=True, alpha=0.5, label='Monte Carlo Scenarios')
```

```
# Plot VaR Lines
```

```
for cl, VaR in VaRs.items():
```

```
    plt.axvline(x=-VaR, color=colors[cl], linestyle='--', linewidth=2, label=f'VaR at {int(cl * 100)}% Confidence')
```

```
plt.xlabel('Scenario Gain / Loss ($)')
```

```
plt.ylabel('Frequency')
```

```
plt.title(f'Distribution of Portfolio {days}-Day Returns and Monte Carlo VaR Estimates')
```

```
plt.legend()
```

```
plt.show()
```

